HOCHSCHULE
CAMPUS WIEN
UNIVERSITY OF
APPLIED SCIENCES

Design document - Medikamentendosierspender

Group Nr. 5

Johannes Graf
Klaus Johannes Rusch
Valentina Krennwallner

Contents

1 Project description **3**

2 Software architecture and design **6**

3 Program flowchart **11**

4 Hazard identification **14**

5 Threat identification **16**

6 Requirements **19**

1 Project description

This project develops a medication dispenser that supports elderly people in taking their medication regularly and correctly. At the scheduled intake time, the system sends a push notification reminding the person of the due dose and then uses a weight sensor to verify whether the correct amount was taken. The result is shown directly on the device via an RGB LED. The system is implemented with an ESP32 as the sensor node and a Raspberry Pi 4B as the gateway, which communicate with each other via encrypted Bluetooth.

1.1 Problem description

Many elderly people have to take several medications at fixed times every day. A typical example is a 71-year-old man living alone. Simple reminders such as an alarm clock or a push notification only ensure that the person knows that a dose is due. However, they do not guarantee that the dose is actually taken correctly. The notification may be overlooked, the dose may be forgotten despite the reminder, or the container may have been filled incorrectly.

The core problem is therefore that knowing is not the same as acting correctly. Incorrect dosing (overdose, underdose or a missed dose) poses a safety-relevant risk to the person's health. In addition, intake times and medication data are sensitive health data that must be protected accordingly.

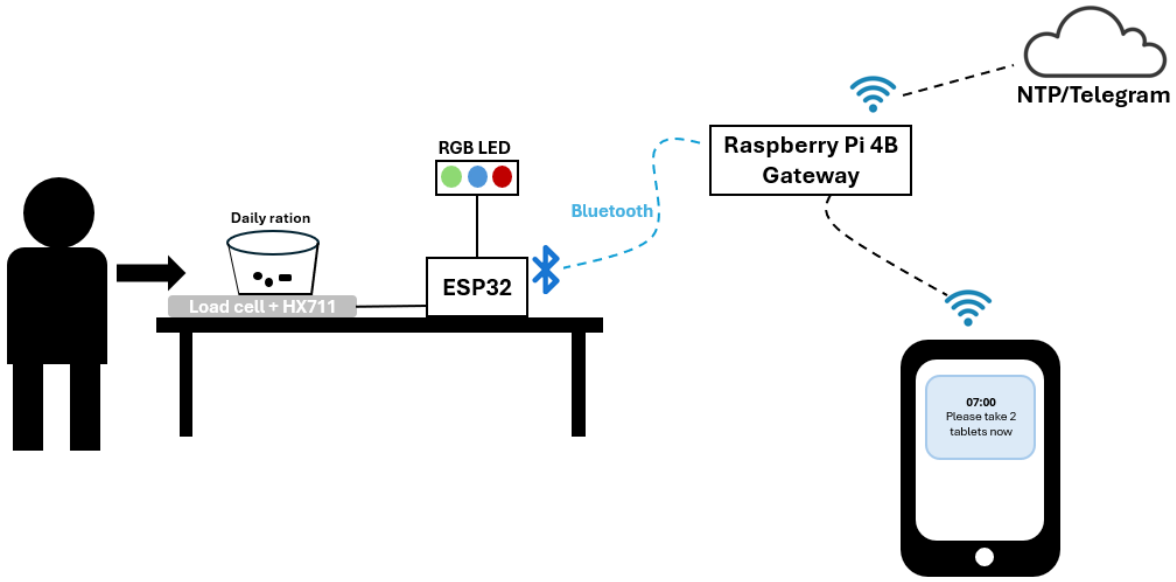


Figure 1: System description

1.2 Proposed solution

The medication dispenser extends the reminder with verified feedback. At the scheduled intake time, the person receives a push notification on their mobile phone stating which medications and how many tablets of each have to be taken (target state). The medication

for the current day is kept in a single container on a weight sensor. Since this container holds the tablets for all intake times of the day, the notification tells the person which tablets to take from it at the current intake time. The weight sensor measures the change in weight when the medication is taken and thus verifies the action that was actually performed (actual state). Since only the weight is measured, the system cannot distinguish between different medications. If, for example, two tablets of 10 g each have to be taken first and one tablet of 20 g later, taking the 20 g tablet first results in the same weight and is not detected (see Hazard 4 in Section 4.2).

The system operates in a daily cycle. Every morning, the nursing staff fills the container with the medication for the current day and starts the dispenser by pressing the reset button (EN) of the ESP32. At start-up, the system checks whether the container holds the correct amount of medication, i.e. whether the measured weight corresponds to the sum of the target weights of all intake times of the day. Each intake window starts at the intake time and lasts 1 h. At the end of the last intake window of the day, the ESP32 checks whether the last dose has been taken, reports the end of the day to the Raspberry Pi and enters deep sleep mode. In this mode, no measurements are performed and the Bluetooth connection is closed. If the last dose has not been taken, the red LED remains on during deep sleep, otherwise the LED is off. The ESP32 only wakes up when the reset button is pressed on the next morning, which starts a new day. The Raspberry Pi keeps running and waits for the next connection.

The system compares the actual state with the target state and displays the result via an RGB LED. In addition, the LED indicates faults of the system. Table 1 lists all states of the LED.

LED	**Meaning**
Green	The remaining weight corresponds to the expected weight (correct dose taken, also after a missed dose has been taken later).
Blue	Deviation from the target weight outside the tolerance, but smaller than the weight of one tablet (e.g. measurement inaccuracy). The result is uncertain.
Red	Critical state (overdose, underdose, missed dose or medication removed outside an intake window).
Red, flashing	Fault due to an incorrect filling at start-up, a sensor fault (see Hazard 2) or a missing valid schedule or time (see Hazard 3). No dose evaluation is performed.
Red, green, blue in sequence	LED self-test at program start.
Off	Within an intake window until the dose is taken and after the last intake window of the day (all doses taken, no fault present).

Table 1: States of the RGB LED

1.3 System overview

The system consists of two microcontrollers (see Figure 1). The ESP32 serves as the sensor node. It reads the load cell via the HX711 amplifier, detects changes in weight, evaluates the dose taken and controls the RGB LED (KY-009). Communication with the Raspberry Pi takes place via Bluetooth. The Raspberry Pi 4B acts as the gateway. It receives the measurement results from the ESP32, synchronises the time via an NTP server and sends push notifications via the Telegram Bot API. The intake schedule is stored in a configuration file on the Raspberry Pi, which is maintained by the nursing staff. For each intake time, it contains the medications and their quantity. From this and the weight of one tablet of each medication, the Raspberry Pi calculates the resulting target weight.

2 Software architecture and design

Building on the system overview in Chapter 1, this chapter describes how the software is divided into modules as well as the libraries, interrupts and pin assignment used. Figure 2 shows the components and communication paths of the system.

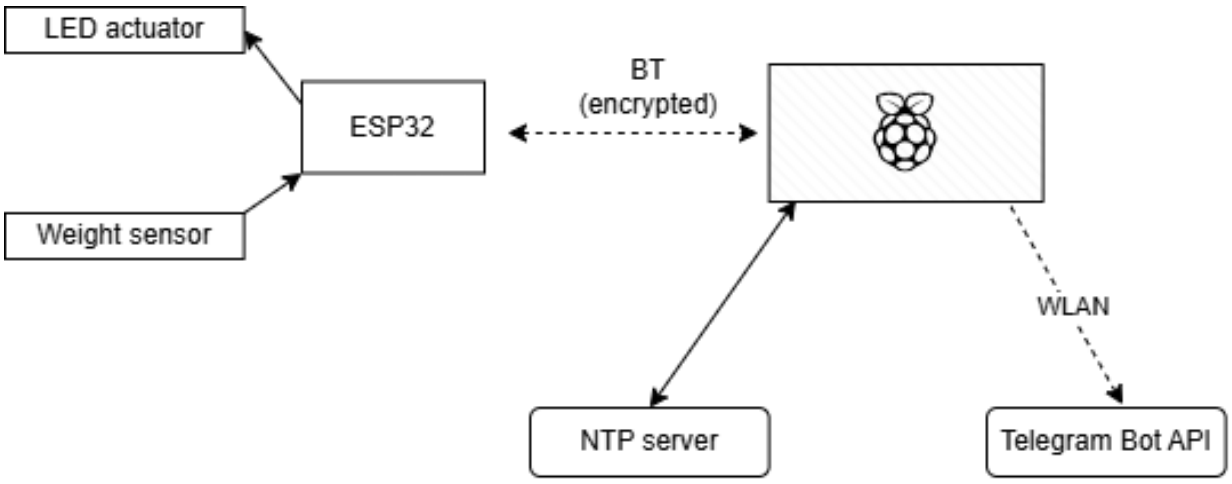


Figure 2: System architecture

The intake schedule is managed on the Raspberry Pi. For each intake time it contains the medications, their quantity and the resulting target weight. Only the intake times and target weights of the current day are transmitted to the ESP32 via Bluetooth together with the current time at start-up; the names of the medications remain on the Raspberry Pi and are only used for the Telegram reminder. The evaluation of the intake takes place locally on the ESP32, so that the LED display also works when the connection is interrupted. The software is implemented in C according to MISRA C and is divided into several threads.

2.1 Software modules

Safety related modules

1. Weight measurement (ESP32):
   Description: Reads the load cell via the HX711, converts the raw values into grams, smooths them and checks them for plausibility. The calibration factor and the tare offset of the empty container are determined once during commissioning using a reference weight and are stored in non-volatile memory (NVS). At start-up, the module checks whether the filling of the container corresponds to the sum of the target weights of the day.
   Functions: weight_read(), weight_calibrate(), weight_check_filling()
   Data: Raw value, calibration factor, tare offset, weight in grams
   Requirements see: 2.1.–2.5., 2.7.–2.10.

2. Dose evaluation and intake monitoring (ESP32):
   Description: Creates a table of the expected weights for the day from the schedule.

In each cycle, it compares the measured weight with the expected weight after the last due intake. If it matches, the dose is considered taken, also outside an intake window (late intake). Otherwise, within an intake window the dose is not yet taken or a wrong amount was removed. Outside an intake window, the deviation is detected as a missed dose or an unscheduled removal.
Functions: table_create(), dose_evaluate()
Data: Table of expected weights, tolerance, status
Requirements see: 1.6.–1.8., 2.6., 2.11.–2.13., 3.2., 3.3.

3. LED status display (ESP32):
Description: Displays the status via the RGB LED and tests all three colour channels at program start.
Functions: led_set_status(), led_self_test()
Data: Current status
Requirements see: 1.6.–1.8., 2.6., 2.10., 2.12., 2.13., 3.3.

4. Intake schedule and time control (Raspberry Pi):
Description: Reads the intake schedule from a configuration file maintained by the nursing staff and calculates the target weight of each intake time (quantity × weight of one tablet). Transmits the intake times and target weights to the ESP32 and triggers the push notification at the intake time, which lists the medications and their quantity. The time is synchronised via NTP.
Functions: schedule_load(), schedule_check_due()
Data: Configuration file (medication names, weight of one tablet, intake times, quantity), calculated target weights, system time
Requirements see: 1.1.–1.5., 1.9.– 1.11., 4.6.

Security related modules

1. Bluetooth communication and encryption (ESP32 and Raspberry Pi):
Description: Establishes the connection, performs an ECDHE key exchange and encrypts all messages with AES-GCM. A sequence number protects against replay attacks. The Raspberry Pi only connects to the BLE address of the ESP32 stored in its configuration file. Both devices hold a common pre-shared key (PSK), which is stored once during commissioning. The session key is derived from the ECDHE shared secret and the PSK, so that only the two known devices can communicate with each other.
Functions: bt_send(), bt_receive()
Data: Pre-shared key, BLE address of the partner, session key, sequence counter, message buffer
Requirements see: 3.1.–3.5., 4.1.–4.3., 4.7.–4.9.

2. Telegram connection (Raspberry Pi):
Description: Sends reminders and warnings via the Telegram Bot API over HTTPS and verifies the server certificate. Before the first NTP synchronisation, the last system time saved at shutdown is used for this check.
Functions: telegram_send()

Data: Bot token, chat ID, message text (medication names and quantity)
Requirements see: 1.2., 1.10., 1.11., 2.6., 2.10., 2.13., 3.5., 4.5., 4.10.

Modules with no influence on safety and security

1. System start and logging (ESP32 and Raspberry Pi):
   Description: Initialises the hardware and modules, starts the threads and outputs status messages without health data.
   Functions: app_main() or main()
   Requirements see: 4.4., 5.1.–5.4.

2.2 Libraries

ESP-IDF with FreeRTOS (ESP32)

- xTaskCreate(pvTaskCode, pcName, usStackDepth, pvParameters, uxPriority, pxCreatedTask) creates a thread.
- gpio_set_level(gpio_num, level) and gpio_get_level(gpio_num) control the HX711 and switch the colour channels of the RGB LED on and off.
- esp_timer_start_periodic(timer, period) starts the periodic timer for the measurement interval and the intake time window.
- portENTER_CRITICAL(mux) and portEXIT_CRITICAL(mux) disable and reenable interrupts while the HX711 is being read.
- ble_gatts_notify_custom(conn_handle, chr_val_handle, om) from NimBLE sends a message to the Raspberry Pi.
- nvs_set_blob(handle, key, value, length) and nvs_get_blob(handle, key, out_value, length) store and read the calibration data in the non-volatile memory.
- rtc_gpio_hold_en(gpio_num) keeps the red LED switched on during deep sleep if a missed dose is displayed.
- esp_deep_sleep_start() puts the ESP32 into deep sleep mode after the last intake window of the day. It only wakes up when the reset button (EN) is pressed.

mbedTLS (ESP32 and Raspberry Pi)

- mbedtls_ecdh_calc_secret(ctx, olen, buf, blen, f_rng, p_rng) computes the shared secret during the key exchange.
- mbedtls_gcm_crypt_and_tag(ctx, mode, length, iv, iv_len, add, add_len, input, output, tag_len, tag) encrypts a message.
- mbedtls_gcm_auth_decrypt(ctx, length, iv, iv_len, add, add_len, tag, tag_len, input, output) decrypts a message and verifies its integrity.

- mbedtls_hkdf(md, salt, salt_len, ikm, ikm_len, info, info_len, okm, okm_len) derives the session key from the ECDHE shared secret and the pre-shared key.

Raspberry Pi

- pthread_create(thread, attr, start_routine, arg) creates a thread.
- curl_easy_perform(handle) from libcurl sends the HTTPS request to the Telegram Bot API.
- clock_gettime(clock_id, tp) reads the system time synchronised via NTP.
- WriteValue(value, options) and StartNotify() are methods of the BlueZ D-Bus interface org.bluez.GattCharacteristic1 and send messages to the ESP32 or enable reception.

For Bluetooth, the NimBLE stack included in ESP-IDF is used on the ESP32 and BlueZ is used on the Raspberry Pi.

The libraries used are available at ESP-IDF `https://github.com/espressif/esp-idf`, mbedTLS `https://github.com/Mbed-TLS/mbedtls`, BlueZ `https://git.kernel.org/pub/scm/bluetooth/bluez.git` and libcurl `https://curl.se/libcurl/`.

2.3 Interrupts

The ESP32 uses a timer interrupt for the measurement interval and the intake time window, as well as interrupts from the Bluetooth stack when messages are received. The interrupt routines only notify the responsible thread. The thread priorities follow the rule of safety before communication before logging (see Table 2).

Thread	Priority
Weight measurement	high
Dose evaluation and LED status display	high
Bluetooth communication	medium
Heartbeat	medium
Logging	low

Table 2: Thread priorities on the ESP32

Reading the HX711 is time-critical, as it enters power-down mode if the clock line remains HIGH for too long (more than 60 µs). It is therefore performed without interruption by other interrupts.

On the Raspberry Pi, interrupts are managed by the operating system (Linux). Bluetooth communication, time control and the Telegram connection each run in their own POSIX thread.

2.4 Pinout

The load cell is connected to the ESP32 via the HX711. The RGB LED is connected directly to three GPIO pins that are configured as digital outputs. Since the KY-009 module has no built-in series resistors, a series resistor is placed in each colour line. The resistors are calculated using $R = (3.3\,\mathrm{V} - U_F)/I$. With a forward voltage U_F of approximately 1.8 V for red and 2.8 V for green and blue, 220 Ω is chosen for red (approx. 7 mA) and 160 Ω for green and blue (approx. 3 mA). These currents are sufficient for a status display and stay well below the maximum current of an ESP32 GPIO pin. Since there are variants of the module in which red and green are swapped, the pin assignment is checked individually before the final wiring (source `https://arduinomodules.info/ky-009-rgb-full-color-led-smd-module/`). Figure 3 shows the complete wiring, Tables 3–5 list the individual connections.

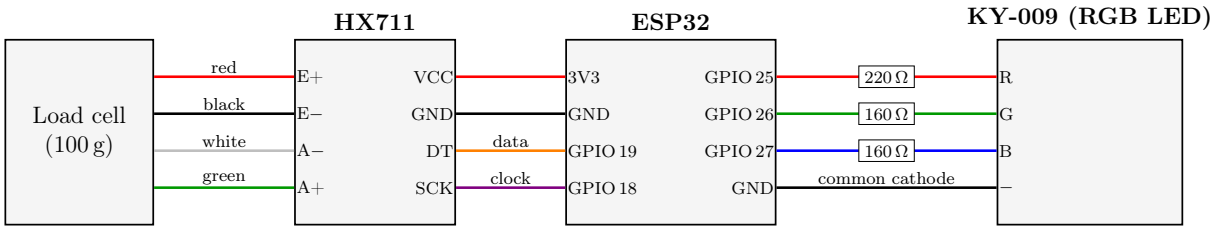


Figure 3: Wiring of the load cell, HX711, ESP32 and RGB LED

Load cell (wire)	HX711
Red	E+
Black	E−
White	A−
Green	A+

Table 3: Pinout load cell and HX711

ESP32 pin	HX711
3V3	VCC
GND	GND
GPIO 19	DT (data)
GPIO 18	SCK (clock)

Table 4: Pinout ESP32 and HX711

ESP32 pin	KY-009
GPIO 25 (via 220 Ω)	R (red)
GPIO 26 (via 160 Ω)	G (green)
GPIO 27 (via 160 Ω)	B (blue)
GND	− (common cathode)

Table 5: Pinout ESP32 and RGB LED (KY-009)

3 Program flowchart

Figure 4 shows the program flow of the ESP32 sensor node and Figure 5 the program flow of the Raspberry Pi gateway. Hazard handling (H1–H3, see Chapter 4) and critical intake results are highlighted in red.

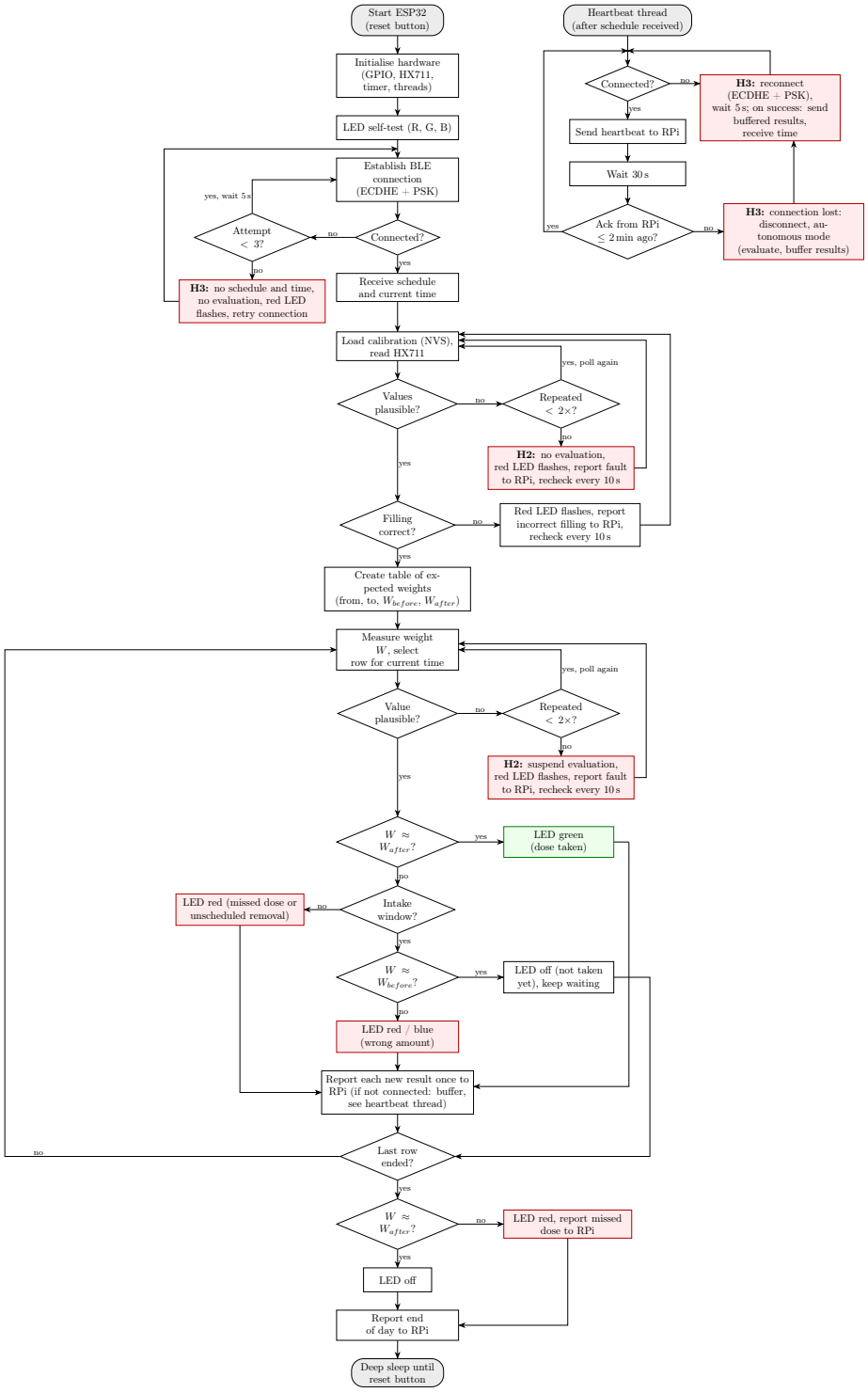


Figure 4: Program flowchart ESP32 (sensor node)

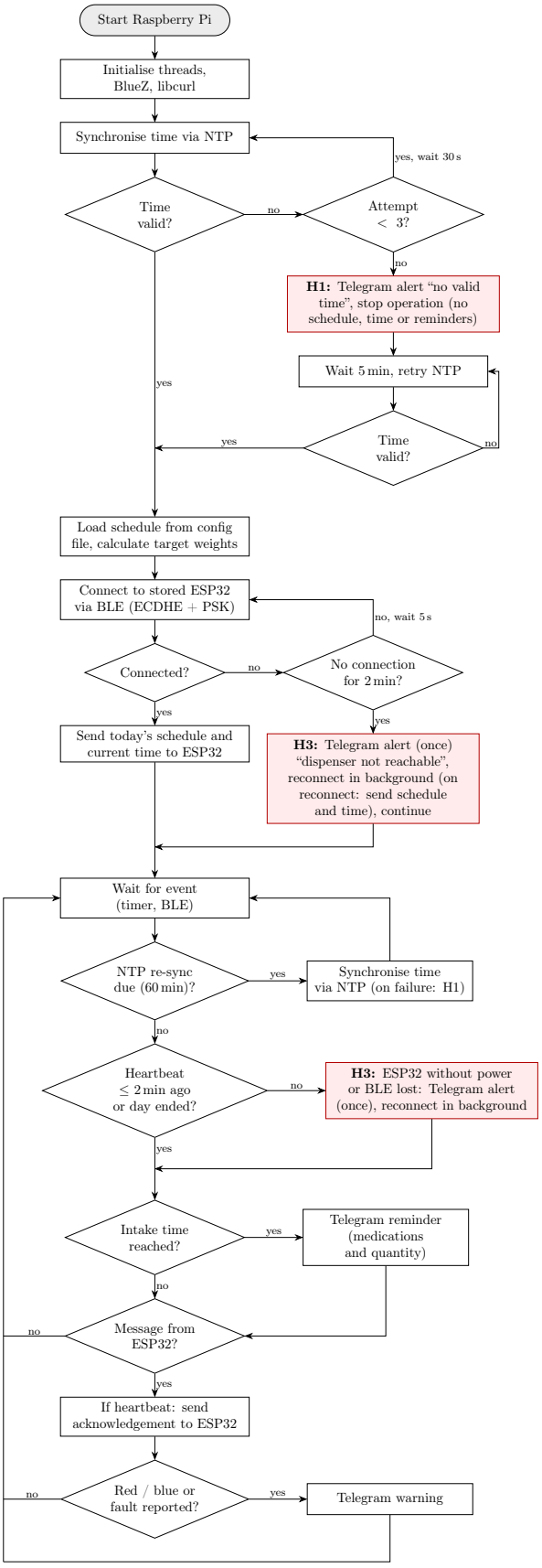


Figure 5: Program flowchart Raspberry Pi (gateway)

For each timeslot, the expected weight before and after intake is calculated. A sample table listed in Table 6.

Phase	Intake [mg]	From	To	W_{before} [mg]	W_{after} [mg]
Initial load			07:00	400	400
First intake	100	07:00	08:00	400	300
Outside intake		08:00	12:00	300	300
Second intake	200	12:00	13:00	300	100
Outside intake		13:00	18:00	100	100
Third intake	100	18:00	19:00	100	0

Table 6: Intake phases and weight changes

4 Hazard identification

4.1 Identified hazards and countermeasures

1. Hazard 1: Raspberry Pi - no connection to NTP server.
   If the Raspberry Pi cannot synchronise its time with the NTP server after three attempts (one attempt every 30 s), it sends an alert via Telegram and stops operation, i.e. it does not send a schedule or time to the ESP32 and no reminders are sent. The synchronisation is retried every 5 min and normal operation resumes once a valid time is available.
   Requirement see 1.

2. Hazard 2: HX711 sensor is not delivering any values or incorrect values.
   A measurement is considered faulty if the HX711 does not deliver a value within 200 ms, the raw value is at the limit of the 24-bit range (broken cable), the weight is below −5 g or above 100 g (capacity of the load cell, e.g. wrong orientation), or the weight changes by more than 5 g outside an intake window (objects added or removed). In this case the sensor is polled two more times. If both measurements are also faulty, the ESP32 suspends the dose evaluation, the red LED flashes and the fault is reported to the Raspberry Pi, which sends an alert via Telegram. The sensor is checked again every 10 s.
   Requirement see 2.

3. Hazard 3: The Bluetooth connection between the ESP32 and the Raspberry Pi is not established or is lost.
   If the ESP32 cannot establish a connection at start-up after three attempts (5 s apart), no valid schedule or time is available. In this case no evaluation is performed, the red LED flashes and the ESP32 keeps trying to connect. If the connection is lost during the day, the ESP32 switches to autonomous mode: it evaluates the intake with the schedule received at start-up, displays the result via the RGB LED and keeps trying to reconnect in the background. The ESP32 sends a heartbeat to the Raspberry Pi every 30 s, which the Raspberry Pi answers with an acknowledgement. If the ESP32 receives no acknowledgement for 2 min, it detects the loss of connection, terminates the connection and switches to autonomous mode. If the Raspberry Pi receives no heartbeat for 2 min, it reports the issue via Telegram. This also detects an ESP32 without power. After the last intake window, the ESP32 reports the end of the day, so that the following deep sleep does not trigger an alert.
   Requirement see 3.

4.2 Identified hazards without countermeasures

1. Hazard 1: Raspberry Pi does not have power.
   No reminders or alerts can be sent via Telegram. If the Raspberry Pi fails during the day, the ESP32 continues in autonomous mode (see Hazard 3) for the rest of the

day, but the person only receives feedback via the LED. Without the Raspberry Pi, the dispenser cannot be started on the next day. A backup power supply is not part of this project.

2. Hazard 2: Mobile phone is switched off.
   The Telegram reminder does not reach the person, so the dose may be forgotten. The system cannot detect whether a message has been received or read.

3. Hazard 3: LED mechanically or electrically damaged.
   The ESP32 has no feedback on whether the LED actually lights up, so a wrong or missing status display cannot be detected. The LED self-test at start-up only allows the person to check the LED visually.

4. Hazard 4: Wrong medication with the same total weight is taken.
   The system only measures the weight removed from the container and cannot distinguish between different medications. If, for example, two tablets of 10 g each have to be taken at the first intake time and one tablet of 20 g at a later intake time, taking the 20 g tablet at the first intake time results in the same weight and is evaluated as a correct dose (LED green). The wrong medication is therefore not detected. The Telegram reminder lists the names and quantities of the medications to be taken, but the correct choice remains the responsibility of the person.

5 Threat identification

5.1 Identified threats and countermeasures

1. Threat 1: Eavesdropping on the Bluetooth connection.
   An attacker within radio range records the Bluetooth traffic between the ESP32 and the Raspberry Pi and reads intake times, target weights and intake results. These are sensitive health data. To prevent this, the two devices perform an ECDHE key exchange each time a connection is established and encrypt all messages with AES-GCM using the resulting session key (see module "Bluetooth communication and encryption" in Section 2.1). As a new session key is agreed for every connection, the recorded traffic does not reveal the content of the messages.
   Requirement see 4.1., 4.2.

2. Threat 2: Manipulation or replay of Bluetooth messages.
   An attacker modifies, injects or replays messages. A modified schedule or target weight would lead to a wrong dose evaluation. A replayed result or heartbeat could hide a missed dose or an ESP32 without power and would suppress the Telegram warning (see Hazard 3). Since all messages are encrypted with AES-GCM, modified messages are detected by the authentication tag and are rejected. In addition, every message carries a sequence number, so that replayed messages are recognised and discarded (see module "Bluetooth communication and encryption" in Section 2.1).
   Requirement see 4.2., 4.3.

3. Threat 3: Disclosure of health data via the log output.
   The log output of the ESP32 and the Raspberry Pi can be read by anyone with access to the serial interface of the ESP32 or the terminal of the Raspberry Pi. If it contained intake times, doses or weights, these health data would be disclosed. Therefore, the module "System start and logging" only outputs status messages (start-up, connection state, faults) without health data.
   Requirement see 4.4.

4. Threat 4: Disruption of the Bluetooth connection (jamming, connection drops).
   An attacker deliberately interferes with the radio channel or provokes repeated connection drops, so that the ESP32 and the Raspberry Pi can no longer communicate. This corresponds to the loss of connection described in Hazard 3 and is handled by the same countermeasures: if the connection is lost during the day, the ESP32 switches to autonomous mode. There it continues to evaluate the intake with the schedule received at start-up, displays the result via the RGB LED, buffers the results and keeps trying to reconnect in the background (see Figure 4). The Raspberry Pi detects the disruption by the missing heartbeat (every 30 s) and sends a one-time Telegram alert after 2 min (see Figure 5). The safety function, i.e. the feedback via the LED, is therefore maintained even when the connection is disrupted.
   Requirement see 3.1.–3.5.

5. Threat 5: Manipulation of the scale outside the intake window.
   A person or an attacker with access to the container places objects on the load cell or removes content in order to falsify the evaluation, or damages the load cell or its

wiring. Such interference outside the intake window is detected. Outside an intake window, the measured weight is compared with the expected weight from the table of expected weights. Any deviation beyond the tolerance is detected as an unscheduled removal: the LED shows red and the Raspberry Pi sends a warning via Telegram. Larger changes are detected as part of Hazard 2: a weight change of more than 5 g outside an intake window as well as implausible values (raw value at the limit of the 24-bit range, weight below −5 g or above 100 g) are considered a faulty measurement. After two further faulty measurements, the ESP32 suspends the dose evaluation, the red LED flashes and the Raspberry Pi sends an alert via Telegram. The sensor is checked again every 10 s.
Requirement see 2.4.–2.7., 2.13.

6. Threat 6: Connection to a foreign device (impersonation, man-in-the-middle).
The ECDHE key exchange alone protects against passive eavesdropping, but not against an attacker who actively intervenes while a connection is being established. A foreign device could impersonate the Raspberry Pi and transmit a modified schedule, or agree on a separate key with each side and thereby read and modify all messages (see Threats 1 and 2). To prevent this, both devices only communicate with their known partner. The Raspberry Pi only connects to the BLE address of the ESP32 stored in its configuration file, and the ESP32 only accepts one connection at a time. In addition, both devices hold a common pre-shared key (PSK), which is stored once during commissioning. The session key is derived from the ECDHE shared secret and the PSK. A device that does not know the PSK therefore derives a different session key, its messages fail the AES-GCM authentication and the connection is terminated. Since the BLE address alone can be spoofed, the PSK is the decisive protection. The ESP32 does not use TLS or X.509 certificates. As the PSK has no period of validity, the ESP32 can establish the secure connection without a valid time. The current time is only received via this authenticated connection.
Requirement see 4.7.–4.9.

7. Threat 7: Connection to a forged Telegram server.
An attacker in the network redirects the HTTPS connection of the Raspberry Pi to a forged server in order to read the reminders and warnings or to suppress them. Therefore, the Raspberry Pi verifies the server certificate of the Telegram Bot API for every HTTPS connection. This check requires a valid time. Before the first NTP synchronisation after start-up (e.g. for the alert of Hazard 1), the Raspberry Pi uses the last system time saved at shutdown, which is sufficient to check the period of validity of the certificate.
Requirement see 4.10.

5.2 Identified threats without countermeasures

1. Threat 1: Physical access to the devices.
A person with physical access to the devices can read the schedule on the ESP32, read the pre-shared key, read or modify the configuration file with the intake schedule or read the Telegram bot token on the Raspberry Pi, exchange the tablets in the container or tamper with the load cell. Protection of the stored data (e.g. flash

encryption, secure boot) and a tamper-proof housing are not part of this project. The system is intended for use in the private home of the person, so physical access is assumed to be controlled by the person.

2. Threat 2: Exposure of health data on the Telegram side.
   The reminders contain the names and quantities of the medications to be taken, as the person needs this information to take the correct tablets from the shared container. The connection to the Telegram Bot API is protected by HTTPS, but the messages are not end-to-end encrypted and are processed by an external service. Furthermore, a third party who gains access to the mobile phone or the Telegram account of the person can read the messages. The system has no influence on this and cannot detect it. The messages are therefore limited to the information necessary for the reminder.

3. Threat 3: Manipulated time through forged NTP responses.
   NTP is an unauthenticated protocol. An attacker in the same network can provide the Raspberry Pi with a wrong but formally valid time. The reminders would then be sent at the wrong time, and the ESP32 would determine the intake window based on the wrong time, as the Raspberry Pi transmits the current time to the ESP32 each time a connection is established (see Requirement 1.4. and Figure 5). The countermeasures of Hazard 1 only take effect if no time is received, not if the time is forged. Protection, e.g. via NTS or several independent time sources, is not part of this project.

4. Threat 4: Circumvention of the verification within the intake window.
   The evaluation by `dose_evaluate()` compares the measured weight with the expected weight from the table of expected weights. Within the intake window, the system therefore cannot distinguish whether the tablets were actually taken, were merely removed, or whether the weight change was caused by other objects. The scale verifies the removal, not the intake (see Section 1.2). Additional detection, e.g. through further sensors, is not part of this project.

6 Requirements

6.1 Safety related requirements

1. Requirement: The Raspberry Pi and the ESP32 must have a consistent, valid time obtained from the NTP server via the Raspberry Pi.
   Folge Requirements: 1.1., 1.2., 1.3., 1.4., 1.5., 1.6., 1.7., 1.8., 1.9., 1.10., 1.11.

   1.1. Requirement: The Raspberry Pi must synchronise its time with the NTP server at start-up and every 60 min.

   1.2. Requirement: If the synchronisation fails three times in a row (one attempt every 30 s), the Raspberry Pi must send an alert via Telegram and stop operation, i.e. it must neither send a schedule or time to the ESP32 nor send reminders.

   1.3. Requirement: While operation is stopped, the Raspberry Pi must retry the synchronisation every 5 min and resume normal operation once a valid time is available.

   1.4. Requirement: The Raspberry Pi must transmit the current time to the ESP32 each time a Bluetooth connection is established.

   1.5. Requirement: The schedule transmitted to the ESP32 must contain the intake times and target weights of the current day.

   1.6. Requirement: At the end of the last intake window of the day, the ESP32 must check whether the measured weight corresponds to W_{after} of the last intake window. If it does not, the LED must show red and a missed dose must be reported to the Raspberry Pi, which must send a warning via Telegram. Otherwise, the LED must be switched off.

   1.7. Requirement: After the check of Requirement 1.6., the ESP32 must report the end of the day to the Raspberry Pi and enter deep sleep mode, in which no measurements are performed and the Bluetooth connection is closed. If a missed dose is displayed, the red LED must remain on during deep sleep. The ESP32 must only wake up when the reset button (EN) is pressed.

   1.8. Requirement: The LED must show green as long as the measured weight corresponds to the expected weight after the last due intake. Within an intake window, the LED must be off until the dose has been taken. If an intake window has ended without the dose being taken (missed dose) or if medication has been removed outside an intake window, the LED must show red. If a wrong amount has been removed within an intake window, the LED must show red, or blue if the deviation is smaller than the weight of one tablet. The red or blue LED must remain on until the measured weight corresponds to the expected weight again, e.g. after the missed dose has been taken.

   1.9. Requirement: At start-up, the Raspberry Pi must read the intake schedule from a configuration file. For each medication, the file contains its name and the weight of one tablet. For each intake time, it contains the medications and their quantity.

1.10. Requirement: The Raspberry Pi must calculate the target weight of each intake time as the sum of quantity × weight of one tablet. If the configuration file is missing or invalid, the Raspberry Pi must send an alert via Telegram and must not transmit a schedule to the ESP32.

1.11. Requirement: At each intake time, the Raspberry Pi must send a Telegram reminder listing the names and quantities of the medications to be taken.

2. Requirement: The ESP32 must detect missing or implausible values of the HX711 sensor and must only use plausible values for the filling check at start-up and for the dose evaluation.
Folge Requirements: 2.1., 2.2., 2.3., 2.4., 2.5., 2.6., 2.7., 2.8., 2.9., 2.10., 2.11., 2.12., 2.13.

2.1. Requirement: A measurement must be considered faulty if the HX711 does not deliver a value within 200 ms.

2.2. Requirement: A measurement must be considered faulty if the raw value equals the minimum or maximum of the 24-bit range (`0x800000` or `0x7FFFFF`).

2.3. Requirement: A measurement must be considered faulty if the weight is below −5 g or above 100 g (capacity of the load cell).

2.4. Requirement: A measurement must be considered faulty if the weight changes by more than 5 g outside an intake window.

2.5. Requirement: After a faulty measurement, the ESP32 must poll the sensor two more times. A sensor fault must only be assumed if both repeated measurements are also faulty.

2.6. Requirement: In case of a sensor fault, the ESP32 must suspend the dose evaluation, let the red LED flash and report the fault to the Raspberry Pi. The Raspberry Pi must then send an alert via Telegram.

2.7. Requirement: During a sensor fault, the ESP32 must check the sensor again every 10 s and resume normal operation once valid values are delivered.

2.8. Requirement: The calibration factor and the tare offset of the empty container must be determined once during commissioning using a reference weight and must be stored in the non-volatile memory of the ESP32. If no calibration data are available at start-up, the ESP32 must start the calibration via the serial interface and must not perform any evaluation.

2.9. Requirement: At start-up, the ESP32 must check whether the measured weight corresponds to the sum of the target weights of the current day within the tolerance.

2.10. Requirement: If the filling is incorrect, the ESP32 must let the red LED flash and report this to the Raspberry Pi, which must send an alert via Telegram. The ESP32 must repeat the check every 10 s and start normal operation once the filling is correct.

2.11. Requirement: After the filling check, the ESP32 must create a table of the expected weights for the day. For each intake window and for each period between

two intake windows, the table contains the start and end time (each intake window starts at the intake time and lasts 1 h) as well as the expected weight before (W_{before}) and after (W_{after}) the intake. Between two intake windows, both values are identical.

2.12. Requirement: The ESP32 must first compare the measured weight with W_{after} of the current row. If it corresponds within the tolerance, the dose must be considered taken (LED green). This also applies outside an intake window, so that a missed dose can be taken later.

2.13. Requirement: If the measured weight does not correspond to W_{after}, the dose must be considered not yet taken within an intake window if the weight corresponds to W_{before} (LED off). Any other weight within an intake window must be considered a wrong amount (LED red, or blue if the deviation is smaller than the weight of one tablet). Outside an intake window, the deviation must be considered a missed dose or an unscheduled removal (LED red). Each new result must be reported once to the Raspberry Pi, which must send a warning via Telegram in case of a deviation.

3. Requirement: The ESP32 and the Raspberry Pi must establish and maintain a Bluetooth connection with each other.
Folge Requirements: 3.1., 3.2., 3.3., 3.4., 3.5.

3.1. Requirement: At start-up, the ESP32 must make three attempts to establish the connection, 5 s apart.

3.2. Requirement: The ESP32 must assume a loss of connection if it has not received an acknowledgement of its heartbeat for 2 min or if the Bluetooth stack reports a disconnection. In this case, it must terminate the connection and switch to autonomous mode: it must evaluate the intake with the schedule received at start-up, display the result via the RGB LED, buffer the results and try to reconnect every 5 s. After reconnecting, it must send the buffered results to the Raspberry Pi and receive the current time.

3.3. Requirement: If no connection can be established at start-up, no valid schedule or time is available. In this case the ESP32 must not perform any evaluation, must let the red LED flash and must keep trying to establish the connection.

3.4. Requirement: While connected, the ESP32 must send a heartbeat to the Raspberry Pi every 30 s. The Raspberry Pi must answer each heartbeat with an acknowledgement.

3.5. Requirement: If the Raspberry Pi receives no heartbeat for 2 min, it must send an alert via Telegram and keep trying to reconnect. After the ESP32 has reported the end of the day, the Raspberry Pi must not monitor the heartbeat until the next connection is established.

6.2 Security related requirements

4. Requirement: The communication between the ESP32 and the Raspberry Pi as well as the output of the system must not disclose or allow manipulation of health data.
Folge Requirements: 4.1., 4.2., 4.3., 4.4., 4.5., 4.6., 4.7., 4.8., 4.9., 4.10.

4.1. Requirement: Each time a Bluetooth connection is established, the ESP32 and the Raspberry Pi must perform an ECDHE key exchange and agree on a new session key.

4.2. Requirement: All messages between the ESP32 and the Raspberry Pi must be encrypted and authenticated with AES-GCM using the session key. Messages with an invalid authentication tag must be discarded.

4.3. Requirement: Every message must carry a sequence number. Messages whose sequence number is not greater than that of the last accepted message must be discarded.

4.4. Requirement: Log messages must not contain any health data (intake times, doses, weights).

4.5. Requirement: Telegram messages must only be sent via HTTPS and must only contain the information necessary for the reminder or warning.

4.6. Requirement: The configuration file on the Raspberry Pi (intake schedule, BLE address of the ESP32, pre-shared key) must only be readable and writable by the user account under which the software runs (file permissions 600).

4.7. Requirement: The ESP32 and the Raspberry Pi must hold a common 256-bit pre-shared key, which is stored once during commissioning (ESP32: non-volatile memory via the serial interface, Raspberry Pi: configuration file).

4.8. Requirement: The Raspberry Pi must only connect to the BLE address of the ESP32 stored in its configuration file. The ESP32 must only accept one connection at a time.

4.9. Requirement: The session key must be derived from the ECDHE shared secret and the pre-shared key. If the first message after the key exchange cannot be authenticated, the connection must be terminated.

4.10. Requirement: The Raspberry Pi must verify the server certificate of the Telegram Bot API for every HTTPS connection. Before the first NTP synchronisation, it must use the last system time saved at shutdown for this check.

6.3 Requirements with no influence on Safety and Security

5. Requirement: The ESP32 and the Raspberry Pi must output status messages (start-up, connection state, faults) via the log output.
Folge Requirements: 5.1., 5.2., 5.3., 5.4.

5.1. Requirement: The ESP32 must output its log messages via the serial interface (USB/UART) using the ESP-IDF logging functions. The Raspberry Pi must output its log messages on the terminal. Log messages are not stored permanently.

5.2. Requirement: Every log message must contain a timestamp, the name of the module and a log level (error, warning, info).

5.3. Requirement: At program start, the ESP32 and the Raspberry Pi must output a start-up message containing the software version.

5.4. Requirement: Logging must run in the thread with the lowest priority (see Table 2) and must not block other threads. If the log output fails, operation must continue without logging.

6. Requirement: The source code must be maintainable and traceable.
   Folge Requirements: 6.1., 6.2.

   6.1. Requirement: The software of both devices must be written in C according to the MISRA C guidelines.

   6.2. Requirement: The source code and this design document must be managed in the shared Git repository of the group.